Teaching a Neural Network the Lorenz System

A physics-informed neural network (PINN) that predicts a chaotic system to better than one part in ten thousand

Written for readers with no background in math, computer science or data science. Every section starts with a plain-words explanation; the equations and technical details follow for readers who want them.

Result in one line
The tuned PINN reproduces the Lorenz trajectory from t = 0 to t = 1 with an average (relative L2) error of about **0.0001%** (1.4×10^{-6}) and a worst single-point error of **5.5×10^{-5}**, meeting the 10^{-4} target on both measures.

Code: github.com/blh119/Lorenz_System_Physical_Neural_Network (scripts/lorenz_pinn.py and scripts/lorenz_pinn_full.py)

Contents

1. The problem in one page

2. The Lorenz system: what we are predicting

3. Getting the right answer the traditional way: Euler and RK4

4. Neural networks in plain words

5. What a PINN is, and why it is useful

6. How we measure “wrong”: the three losses and the error scores

7. The knobs: dt, samples, learning rate, epochs and the rest

8. How the network learns: gradient descent, SGD and Adam

9. L-BFGS: the finishing tool, explained in full

10. What we tried and what we found

11. Reading the graphs

12. Conclusions, limits and next steps

13. Glossary

14. How to reproduce the results

15. Validation: training on part of the time window

16. Prediction horizon: how far out, and how much data

1. The problem in one page

In plain words

Imagine a leaf caught in a swirling wind. If you know exactly where it starts and exactly how the wind pushes it, you can in principle work out where it will be at every moment. The Lorenz system is a famous mathematical version of that: three numbers (called x, y and z) that change over time according to three simple rules. Even though the rules are simple, the path they trace is wild and swirling, and tiny changes at the start grow into huge differences later. This sensitivity is called **chaos**.

Our goal was to train a computer program called a **neural network** to predict that path. The twist is that the network is not only shown example points from the path, it is also taught the three rules themselves. That combination is what makes it a **physics-informed neural network**, or PINN.

The target was accuracy within $\mathbf{10^{-4}}$, which means the prediction should be off by less than 1 part in 10,000. For comparison, that is like measuring the length of a football field and being off by less than the thickness of a credit card.

What exactly was asked

- Predict x, y and z for every moment between time t = 0 and t = 1, starting from x = y = z = 1.
- Use the standard Lorenz settings: σ (sigma) = 10, ρ (rho) = 28, β (beta) = 8/3.
- Reach an error below 10^{-4} compared with the true path.

Why only from t = 0 to t = 1?

Because the system is chaotic, any small error doubles roughly every 0.77 time units (its so-called Lyapunov exponent is about 0.9). Over a short window, errors stay small enough to control. Over long windows, even the best methods eventually drift away from the true path, and a single neural network has to describe more and more twists. Keeping T = 1 makes the 10^{-4} target realistic.

The short story of what happened

1. The first version trained the network on a path calculated with **Euler's method**. That path was itself about 0.3% wrong, so the network could never be better than about 0.2%.
2. We switched the “answer key” to a far more accurate method called **RK4**, made the computer use more decimal places (float64), and built the starting point directly into the network.
3. We compared learning settings (learning rate, number of epochs) and two learning methods (**Adam** and **stochastic gradient descent**).
4. We added a second, more precise learning stage called **L-BFGS**. That stage is what pushed the error from about 10^{-4} down to about 10^{-6}.
5. The final tuned model meets the target on every error measure we checked.

2. The Lorenz system: what we are predicting

In plain words

In 1963 the meteorologist Edward Lorenz wrote down a very simplified model of air rising and falling in a heated layer of atmosphere. It boils down to three quantities that change over time. Think of x as how fast the air is rolling, and y and z as how the temperature is distributed. The three rules below say how fast each quantity is changing right now, given where all three are right now.

"How fast something is changing" is called its **rate of change** or **derivative**. It is written dx/dt, which you can read as "the change in x per tiny bit of time."

The equations

$$\frac{dx}{dt} = \sigma(y - x)$$

$$\frac{dy}{dt} = x(\rho - z) - y = \rho x - y - xz$$

$$\frac{dz}{dt} = xy - \beta z$$

With σ = 10, ρ = 28, β = 8/3 and the starting point (x, y, z) = (1, 1, 1) at t = 0.

Reading the equations in words

- **dx/dt = σ(y – x)**: x is always pulled toward y. The bigger the gap, the faster it moves. σ sets how strong that pull is.
- **dy/dt = x(ρ – z) – y**: y grows when x is large and z is below ρ, and it naturally decays (the –y part).
- **dz/dt = xy – βz**: z grows when x and y have the same sign and decays at a rate set by β.

Because each rule depends on the other two variables, the three are tangled together. The terms xz and xy (two variables multiplied) make the system **nonlinear**, which is what allows chaos. No formula gives x(t), y(t) and z(t) directly, so the path has to be computed step by step, or learned.

The famous butterfly

Plotted in 3D, the long-run path loops around two lobes that look like butterfly wings. This is the "Lorenz attractor," and it is where the phrase "butterfly effect" comes from. In our window (t = 0 to 1) the path makes one big swing from the start point into one of the lobes; see the 3D figures in Section 11.

Key takeaway

We are not predicting data from the real world. We are predicting the solution of three known equations. That means we can compute an extremely accurate "true" answer to grade the network against.

3. Getting the right answer the traditional way: Euler and RK4

In plain words

To know if the network is right, we need the true path. The traditional way to get it is to walk forward in tiny time steps. At each step you look at the rules to see which direction things are heading, take a small step in that direction, and repeat. The size of each step is called **dt** (for “delta time”). In this project dt = 0.0001, so getting from t = 0 to t = 1 takes 10,000 steps.

Imagine walking along a curvy path in the dark, checking a compass to decide which way to go. **Euler's method** checks the compass once and then walks straight for the whole step. Because the path curves, you drift a little off every step, and the drift adds up. **RK4** checks the compass four times per step (at the start, twice around the middle, and at the end) and walks in a weighted average of those directions, so it follows the curve instead of cutting corners.

Euler's method

Write u for the three numbers (x, y, z) together, and f(u) for the three right-hand sides of the Lorenz equations. Euler's step is:

$$u_{n+1} = u_n + dt \cdot f(u_n)$$

Each step makes an error proportional to dt^2. Over the 1/dt steps needed to cover the interval, those add up to a total error proportional to **dt**. That is called a **first-order** method: making dt 10 times smaller makes the error only 10 times smaller.

The fourth-order Runge–Kutta method (RK4)

$$k_1 = f(u_n), \quad k_2 = f\left(u_n + \tfrac{dt}{2}k_1\right), \quad k_3 = f\left(u_n + \tfrac{dt}{2}k_2\right), \quad k_4 = f(u_n + dt\,k_3)$$

$$u_{n+1} = u_n + \tfrac{dt}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

The four slopes k_1 to k_4 are combined so that errors cancel up to a high order. The total error is proportional to $\mathbf{dt^4}$: making dt 10 times smaller makes the error about **10,000 times** smaller. RK4 costs four rule evaluations per step instead of one, which is a very good trade.

What we measured

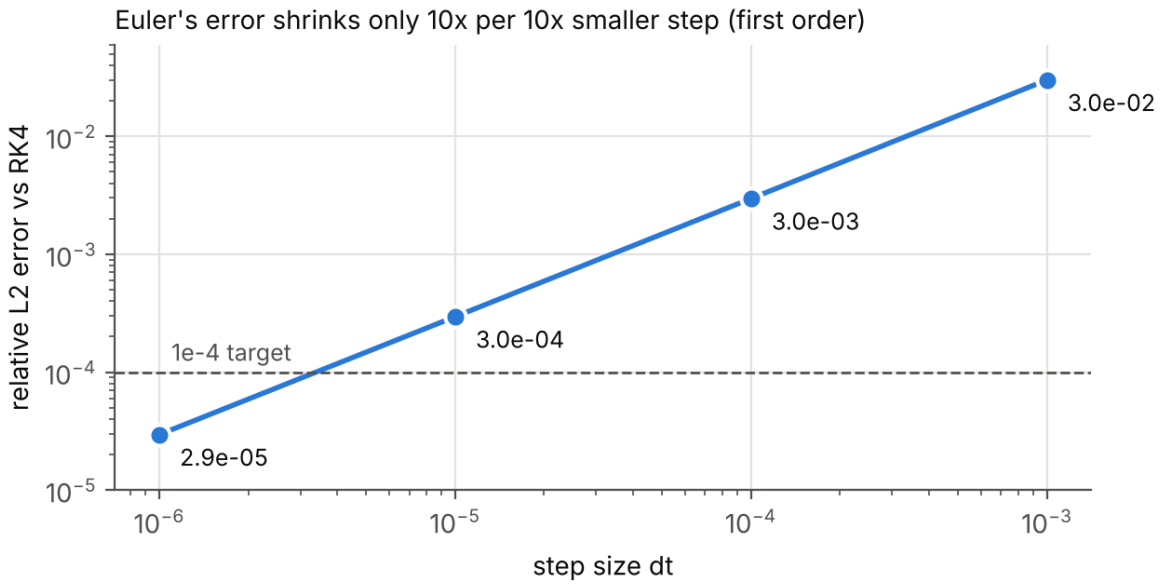


Figure 1. Euler's error compared with RK4 at the same step size, from t = 0 to 1. Each 10× smaller step cuts the error exactly 10×, the signature of a first-order method.

Step size dt	Euler steps	Euler relative error	Euler worst point	RK4 error (estimated)
0.001	1,000	3.0×10^{-2} (3%)	0.77	about 4×10^{-9}
0.0001	10,000	3.0×10^{-3} (0.3%)	0.076	below 10^{-12}
0.00001	100,000	3.0×10^{-4}	0.0076	far below
0.000001	1,000,000	3.0×10^{-5}	0.00076	far below

The RK4 column is estimated by comparing RK4 at dt = 0.001 with RK4 at dt = 0.0001: their final points agreed to about 4×10^{-9}. Even Euler at a million steps is still 7.6×10^{-4} off at its worst point, above the target.

Why this mattered for the PINN

The network learns from example points taken from the reference path. If those points come from Euler at dt = 0.0001, they are already off by about 0.3%. A network that copies them faithfully inherits the error, like a great student copying a flawed answer key. Section 10 shows this directly: every network trained on Euler data stopped at about 1.8×10^{-3}, whatever learning settings were used.

Key takeaway

Euler's method in the original code had no bug. The method itself is simply too rough for a 10^{-4} target. RK4 at the same step size is accurate to better than 10^{-8}, so it serves as the “truth” for both training and grading.

4. Neural networks in plain words

In plain words

A neural network is a very flexible formula with thousands of adjustable dials, called **weights**. You feed in a number (here, a time t) and it produces numbers (here, guesses for x, y and z). At first the dials are set randomly, so the guesses are nonsense.

Training means repeatedly nudging the dials so the guesses get better. To know which way to nudge, we need a single score that says how wrong the network is right now. That score is called the **loss**. Training is simply: lower the loss, a little at a time, many times.

Think of it as a sound engineer with a mixing desk of 12,803 sliders, adjusting them a hair at a time until the recording sounds right.

The network used here

Part	What it is	Why
Input	One number: time t, rescaled from [0, 1] to [–1, 1]	Networks learn best when inputs are small, centered numbers.
Hidden layers	4 layers of 64 “neurons” each, using the tanh function	tanh is smooth, so the network's output has clean derivatives. The physics check needs them.
Output	Three numbers, rescaled to the typical size of x, y and z	z reaches about 48 while x stays near 20; scaling stops z from dominating.
Total dials	12,803 weights	Enough flexibility to bend into the shape of the Lorenz path.
Number precision	float64 (about 16 significant digits)	float32 has about 7 digits, which leaves little room below 10^{-4}.

How a neuron works, briefly

Each neuron takes the numbers coming in, multiplies each by a weight, adds them up with an extra offset (a **bias**), and passes the total through a smooth squashing function (tanh, which maps any number into the range –1 to 1). Stacking many of these lets the network form very complicated shapes. A well-known theorem (the universal approximation theorem) says a large enough network of this kind can approximate any smooth curve as closely as you like. The hard part is finding the right weights, which is what training does.

Built-in starting point (“hard initial condition”)

Instead of hoping the network learns to start at (1, 1, 1), the code builds it in:

$$\hat{u}(t) = u_0 + \frac{t}{T}\,\sigma_{\text{scale}} \odot N\left(\frac{2t}{T} - 1\right)$$

Here N is the raw network, u_0 = (1, 1, 1) and σ_{scale} is the typical size of each variable. At t = 0 the factor t/T is zero, so the output is exactly (1, 1, 1) no matter what the weights are. The network only has to learn how the path moves away from the start.

5. What a PINN is, and why it is useful

In plain words

An ordinary neural network learns only from examples. Show it 200 points from the path and it learns to pass through them, but between those points it can do anything it likes.

A **physics-informed neural network** is also graded on whether it obeys the rules. At thousands of random moments in time, we ask: “Is the network's curve changing at the rate the Lorenz equations say it should?” Any mismatch adds to the loss. So the network is pushed to be both close to the data and physically sensible everywhere in between.

It is like teaching a student with worked examples *and* the textbook rules, and marking them down for any answer that breaks the rules, not just for answers that differ from the examples.

The key trick: automatic differentiation

To check the rules, we need dx/dt for the network's output. Software like TensorFlow can compute that exactly (not approximately) by tracking every operation inside the network and applying the chain rule of calculus automatically. This is called **automatic differentiation**. In the code it is the `GradientTape` in `physics_residual`.

Why PINNs are useful

- **Little data needed.** The physics fills the gaps. Here 200 data points (2% of the 10,001 reference points) were enough, because the equations were checked at thousands more.
- **A smooth, continuous answer.** The trained network can be asked for x, y, z at any time, for example t = 0.123456, not only at the grid points a step-by-step solver visits. Its derivatives are available exactly too.
- **Noisy or partial data.** Because the physics acts as a guide, PINNs can cope with measurements that are noisy or only cover some variables.
- **Inverse problems.** If the equation's constants are unknown, the PINN can learn them from data. The code supports this: with `inverse=True`, σ, ρ and β become learnable.
- **No grid.** In problems with space as well as time (fluids, heat, waves), PINNs avoid building a computational mesh, which can be hard for complicated shapes.

Honest limits

- For a forward problem like this one, where the equations and starting point are fully known, RK4 is far faster: it computes the reference in about a second, while the PINN needs minutes of training. The value of this project is in understanding and validating the PINN approach.
- Chaos makes long time windows very hard for PINNs. Training would need tricks such as splitting time into segments.

- Training is sensitive to settings (learning rate, loss weights, optimizer), which is why Sections 7 to 10 matter.

Where the idea comes from

The modern form was introduced by Raissi, Perdikaris and Karniadakis in 2019 (Journal of Computational Physics). Since then PINNs have been applied to fluid flow, heat transfer, materials, biology and finance.

6. How we measure “wrong”: the three losses and the error scores

In plain words

There are two different kinds of “wrongness” in this project, and it helps to keep them apart.

The **loss** is what the network sees during training. It is the score it tries to lower. It is built from three parts: matching the data, obeying the physics, and starting in the right place.

The **error** is what we use afterwards to judge the result. We compare the trained network with the RK4 truth at all 10,001 time points. The 10^{-4} goal refers to the error, not the loss.

Loss 1: data loss

How far the network is from the 200 sampled reference points. Each difference is divided by that variable's typical size (its standard deviation, written s_v) so that x, y and z count equally, then squared and averaged:

$$L_{\text{data}} = \frac{1}{3N_d}\sum_{i=1}^{N_d}\sum_{v\in\{x,y,z\}}\left(\frac{\hat{u}_v(t_i)-u_v(t_i)}{s_v}\right)^2$$

Squaring makes all differences positive and punishes big misses much more than small ones. The 200 times are chosen by **stratified sampling**: the interval is cut into 200 equal slices and one point is picked from each, so there are no large gaps.

Loss 2: physics loss

At N_c = 2,000 random times (“collocation points”, drawn fresh on every training step) the code computes the **residual**, the gap between the network's actual rate of change and the rate the Lorenz equations demand:

$$r(t) = \frac{d\hat{u}}{dt} - f(\hat{u}(t)), \qquad L_{\text{phys}} = \frac{1}{3N_c}\sum_{j=1}^{N_c}\sum_{v}\left(\frac{T}{2}\,\frac{r_v(t_j)}{s_v}\right)^2$$

If the network obeyed the equations perfectly, every residual would be zero. The factor T/2 and the division by s_v put the residual in the same units as the data loss, because raw rates of change can be in the hundreds and would otherwise drown out everything else.

Loss 3: initial-condition (IC) loss

$$L_{\text{ic}} = \frac{1}{3}\sum_{v}\left(\frac{\hat{u}_v(0)-u_v(0)}{s_v}\right)^2$$

This measures how far the network starts from (1, 1, 1). Because the starting point is now built into the network (Section 4), this loss is always exactly 0. It is kept in the code so the older soft version can still be switched on (`hard_ic=False`).

The total loss and the warm-up

$$L = w_d L_{\text{data}} + w_p(k) L_{\text{phys}} + w_{ic} L_{\text{ic}}, \qquad (w_d, w_p, w_{ic}) = (1, 1, 10)$$

The physics weight w_p is not switched on all at once. For the first 1,000 training steps it is 0 (the network learns the rough shape from data only). Between steps 1,000 and 2,000 it ramps linearly from 0 to 1. Turning the physics on at full strength immediately tends to trap the network in a smooth, wrong curve that roughly obeys the rules but ignores the data.

The error scores used to judge results

Name	What it measures	Formula
Relative L2 error	Average-style error, as a fraction of the path's size. Reported per variable; the worst of x, y, z is quoted.	||pinn – truth|| / ||truth|| over all 10,001 points
Max absolute error	The single biggest miss anywhere, in any variable. The strictest measure.	max |pinn – truth|
3D distance error (err_3d)	Straight-line distance between predicted and true point in (x, y, z) space, at each time.	√(x_err² + y_err² + z_err²)
Combined relative L2	Relative L2 using the 3D distance, treating x, y, z as one position.	||err_3d|| / ||position||

||·|| means the square root of the sum of squares (the “L2 norm”), a standard way to measure the size of a list of numbers.

7. The knobs: dt, samples, learning rate, epochs and the rest

In plain words

A **hyperparameter** is a setting chosen by a person before training, as opposed to the weights, which the training finds. Think of them as the oven temperature and baking time, rather than the cake itself.

Knob	Final value	What it does, in plain words
dt (time step)	0.0001	Step size of the RK4 reference solver: 10,000 steps from t = 0 to 1. Smaller is more accurate but slower.
T (time window)	1.0	How far ahead we predict. Kept short because of chaos.
n_sample	200	How many reference points the network is shown (data loss).
n_colloc	2,000	How many random times the physics is checked on each Adam step.
n_colloc_lbfgs	4,000	Fixed, evenly spaced physics check points used during L-BFGS.
hidden_layers, units	4, 64	Size of the network (12,803 weights).
optimizer	adam	The learning method for the first stage (Section 8).
learning_rate	0.01	How big each nudge to the weights is at the start.
lr_final_fraction	0.01	The learning rate shrinks smoothly to 1% of its start by the end.
epochs	10,000	How many Adam steps are taken.
warmup_epochs	1,000	Data-only steps before the physics is phased in.
loss_weights	(1, 1, 10)	Importance of data, physics and start-point losses.
lbfgs_iters	20,000	Upper limit on L-BFGS steps (Section 9).
hard_ic	True	Build the starting point into the network.
method	rk4	Which solver makes the reference data.
seed	99	Fixes the random starting weights and sampling, so runs can be repeated.

dt: the step size

dt controls how accurate the reference “answer key” is (Section 3). With RK4, dt = 0.0001 is already accurate to better than 10^{-8}, so making it smaller would not help the PINN. All training runs in this project used dt = 0.0001.

Learning rate

In plain words

The learning rate is the size of each nudge to the dials. Too small and learning crawls; too big and the network overshoots, bounces around, or blows up entirely (the numbers grow until the computer reports “NaN”, not a number). The best value depends on the learning method.

The code uses a **cosine schedule**: big steps early to find the right area quickly, small steps late to settle in precisely. With starting rate η_0, step k out of K, and $\alpha = 0.01$:

$$\eta_k = \eta_0\left[(1-\alpha)\,\tfrac{1}{2}\left(1+\cos\tfrac{\pi k}{K}\right)+\alpha\right]$$

So with $\eta_0 = 0.01$, the rate glides from 0.01 down to 0.0001 along a smooth curve. Measured effect (Adam, 10,000 epochs, no L-BFGS): $\eta_0 = 10^{-4}$ gave error 1.4×10^{-2}; 10^{-3} gave 1.9×10^{-4}; 10^{-2} gave 6.5×10^{-5}.

Epochs

An **epoch** here means one training step: compute the loss, compute which way to nudge every dial, nudge them. (In other machine-learning settings an epoch means one pass through a whole dataset; here every step already uses all 200 data points, so the two meanings coincide.) More epochs give more chances to improve, with diminishing returns: in one run the loss fell from 2.0 to 7×10^{-5} in the first 5,000 epochs, then only to 2.7×10^{-5} in the next 5,000.

Collocation points

These are the times where the physics is checked. More points give a more thorough check but cost more per step. During Adam they are re-drawn randomly every step, so over thousands of steps the whole interval is covered. During L-BFGS they are fixed (an evenly spaced grid of 4,000), because L-BFGS needs the loss to be the same function every time it looks (Section 9).

Precision (float32 vs float64)

Computers store numbers with a fixed number of digits. float32 keeps about 7 significant digits, float64 about 16. Because we chase errors around 10^{-6}, and the physics loss involves derivatives and many additions, float64 keeps rounding noise far below the signal. The cost is roughly twice the computation time.

8. How the network learns: gradient descent, SGD and Adam

In plain words

Picture the loss as a hilly landscape, where your position is the setting of all 12,803 dials and your altitude is the loss. Training is trying to walk to the lowest valley, in fog, so you can only feel the slope under your feet.

The slope in every direction at once is called the **gradient**. It points uphill, so we step the opposite way. The three methods below differ in how they turn that slope into a step.

Gradient descent (GD)

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k)$$

θ (theta) stands for all the weights, η (eta) is the learning rate and ∇L is the gradient. Simple and reliable, but slow when the valley is long and narrow: the steps bounce from wall to wall instead of moving along the floor. In PINNs the landscape is often exactly like that, because the physics loss is very sensitive in some directions and almost flat in others (it is **ill-conditioned**).

Stochastic gradient descent (SGD)

Stochastic means “involving randomness.” SGD uses the same update as GD, but the gradient is computed from a random sample instead of everything. In this code the randomness comes from the 2,000 physics points, which are re-drawn every step. The code's `optimizer="sgd"` uses Keras's SGD with no momentum, which is plain stochastic gradient descent.

Adam (adaptive moment estimation)

In plain words

Adam is gradient descent with two upgrades. First, **momentum**: it keeps a running average of recent slopes, like a ball rolling downhill that does not stop at every bump, so the zig-zags cancel out. Second, **a separate step size for every dial**: dials whose slopes are usually large and noisy get smaller steps, and dials with small, steady slopes get relatively bigger ones.

$$m_k = \beta_1 m_{k-1} + (1-\beta_1)\, g_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2)\, g_k^2$$

$$\hat{m}_k = \frac{m_k}{1-\beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1-\beta_2^k}, \quad \theta_{k+1} = \theta_k - \eta_k \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}$$

g_k is the gradient at step k; m is the running average of slopes (momentum); v is the running average of squared slopes (how big the slopes usually are). The hats correct a bias at the start, when the averages begin at zero. Keras defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-7}$. Adam was introduced by Kingma and Ba in 2014.

What we found: Adam vs SGD

Method and learning rate	Epochs	Relative L2 error (no L-BFGS)	Comment
Adam, 0.01	10,000	6.5×10^{-5}	Best; under target on its own
Adam, 0.001	10,000	1.9×10^{-4}	Good
Adam, 0.0001	10,000	1.4×10^{-2}	Too slow
SGD, 0.005	10,000	4.6×10^{-2}	Best SGD
SGD, 0.003	10,000	5.4×10^{-2}	
SGD, 0.01	3,000 / 10,000	2.3×10^{-1} / blew up	Unstable over time
SGD, 0.1	3,000	blew up	Far too large
SGD, 0.001	10,000	8.6×10^{-1}	Barely learns

Adam was about 1,000 times more accurate than the best SGD setting in the same number of steps, and it worked across a wide range of learning rates. SGD only worked in a narrow band (roughly 0.003 to 0.005): lower and it barely moved, higher and it blew up. This matches the ill-conditioning described above: one fixed step size cannot suit both the steep and the flat directions, while Adam adjusts per dial.

Key takeaway

If the requirement is strictly plain gradient descent, SGD at about 0.005 works but stops near 5×10^{-2} without help. Adam at 0.01 reaches 6.5×10^{-5} on its own. Either one followed by L-BFGS reaches about 10^{-5} or better.

9. L-BFGS: the finishing tool, explained in full

In plain words

Adam and SGD only feel the slope under their feet. **L-BFGS** also remembers how the slope changed over its last several steps, and from that it works out the *shape* of the valley: whether it is a narrow ditch or a wide bowl, and which way it bends. Knowing the shape, it can estimate where the bottom is and jump most of the way there in one step. Before committing, it tries the step and checks that the loss really went down, shortening the step if needed.

Adam is like a hiker taking careful steps downhill. L-BFGS is like a surveyor who measures the curvature of the ground and walks straight to the low point. Near the bottom of a valley, where the ground is almost flat and the hiker's steps become tiny, the surveyor is dramatically faster.

9.1 The name

L-BFGS stands for **Limited-memory Broyden–Fletcher–Goldfarb–Shanno**. Charles Broyden, Roger Fletcher, Donald Goldfarb and David Shanno each published the BFGS method independently in 1970. Jorge Nocedal (1980) and Dong Liu with Nocedal (1989) developed the limited-memory version. The code uses **L-BFGS-B** (Byrd, Lu, Nocedal and Zhu, 1995), a variant that also allows upper and lower limits on each weight; we set no limits, so it behaves as plain L-BFGS. It is called through SciPy's `scipy.optimize.minimize(method="L-BFGS-B")`.

9.2 Starting point: Newton's method

If you knew both the slope (gradient g) and the curvature (the **Hessian** matrix H, which describes how every slope changes as every dial moves), the ideal step toward a bowl's bottom is Newton's step:

$$\theta_{k+1} = \theta_k - H_k^{-1} g_k$$

For a perfect bowl this lands exactly at the bottom in one step. Near a minimum, Newton's method is **quadratically convergent**: the number of correct digits roughly doubles each step. The problem: with n = 12,803 weights the Hessian has $n^2 \approx 164$ million entries (about 1.3 GB in float64), it is expensive to compute, and it must be inverted. That is impractical.

9.3 Quasi-Newton: learning the curvature from the slopes

BFGS never computes the Hessian. It watches how the gradient changes between steps. After each step it records two vectors:

$$s_k = \theta_{k+1} - \theta_k \quad \text{(how the weights moved)}, \qquad y_k = g_{k+1} - g_k \quad \text{(how the slope changed)}$$

For a bowl-shaped function, H s = y. BFGS keeps an estimate B of the Hessian (or directly its inverse, written here as H) and updates it so that the newest pair satisfies this **secant condition**, $B_{k+1}s_k = y_k$, while changing the estimate as little as possible. The update for the inverse is:

$$H_{k+1} = \left(I - \rho_k s_k y_k^\top\right) H_k \left(I - \rho_k y_k s_k^\top\right) + \rho_k s_k s_k^\top, \qquad \rho_k = \frac{1}{y_k^\top s_k}$$

If $y_k^T s_k > 0$ (the slope increased in the direction we moved, i.e. the ground curves upward like a bowl), this update keeps H **positive definite**, which guarantees that −Hg (H times the slope g) always points downhill. Full BFGS is **superlinearly convergent** near a smooth minimum: faster than any fixed rate, approaching Newton's speed.

9.4 The limited-memory idea

Full BFGS still stores an n × n matrix. L-BFGS throws the matrix away and keeps only the last m pairs (s_i, y_i). Our code uses m = 50 (`maxcor=50`). Storage: 2 × 50 × 12,803 ≈ 1.3 million numbers (about 10 MB) instead of 164 million. Whenever a direction is needed, the effect of the BFGS matrix on the gradient is rebuilt on the fly from those pairs using the **two-loop recursion**:

```
q ← g_k (current gradient)
for i = k−1 down to k−m: α_i ← ρ_i s_i^T q ; q ← q − α_i y_i
r ← γ q, with γ = s_{k−1}^T y_{k−1} / y_{k−1}^T y_{k−1} (initial curvature guess)
for i = k−m up to k−1: β ← ρ_i y_i^T r ; r ← r + s_i(α_i − β)
search direction d_k = −r
```

This costs about 4mn multiplications (around 2.6 million here), which is tiny next to evaluating the loss and its gradient through the network.

9.5 Line search: how far to step

Having a direction d, L-BFGS picks a step length a along it. It starts by trying a = 1 (the full quasi-Newton step) and adjusts until two **Wolfe conditions** hold:

$$L(\theta + a\,d) \le L(\theta) + c_1\,a\,g^\top d \qquad \text{(sufficient decrease)}$$

$$\left|\nabla L(\theta + a\,d)^\top d\right| \le c_2 \left|g^\top d\right| \qquad \text{(curvature)}$$

The first says the loss must drop by at least a small fraction of what the slope promised (typically $c_1 = 10^{-4}$). The second says the slope must have flattened enough that we did not stop too early (typically $c_2 = 0.9$). The curvature condition also guarantees $y^T s > 0$, which keeps the BFGS estimate valid. Each trial step costs one full loss-and-gradient evaluation, which is why L-BFGS steps are more expensive than Adam steps; SciPy allows up to 20 trials per step (`maxls`).

9.6 When it stops

Stopping rule (SciPy option)	Our setting	Meaning
maxiter	lbfgs_iters = 20,000	Maximum number of L-BFGS steps.
maxfun	15,000 (SciPy default)	Maximum number of loss-and-gradient evaluations. **This is what stopped the tuned run, after 13,757 steps.**
ftol	10^{-16}	Stop if the relative drop in loss between steps is below this. Set tiny so it does not stop early.
gtol	10^{-12}	Stop if the largest slope component is below this, meaning we are at a flat point. Also set tiny.
maxcor	50	Number of (s, y) pairs remembered (m).
maxls	20 (default)	Maximum trial step lengths per line search.

9.7 Why it needs a fixed loss

L-BFGS builds its picture of the valley from differences between gradients at different points. If the loss itself changed between evaluations, those differences would be mostly noise and the curvature estimate would be garbage. That is why the code switches from 2,000 randomly re-drawn physics points (fine for Adam) to a **fixed, evenly spaced grid of 4,000 points** for L-BFGS, and sets the physics weight to its full value. It is also why L-BFGS is a **full-batch** method, unlike stochastic methods.

9.8 Why Adam first, then L-BFGS

- **L-BFGS is a local method.** Its curvature model is only accurate near a minimum. Starting from random weights, far from any good answer, the landscape is bumpy and not bowl-shaped, so its model misleads it and it can stall in a poor spot.
- **Adam is robust far away.** Its momentum and per-weight steps handle the rough early terrain and find the right valley.
- **L-BFGS is precise up close.** Near the bottom, where Adam's steps become tiny and noisy, L-BFGS uses curvature to take large, well-aimed steps. This two-stage recipe is standard in PINN research, and studies of PINN training (for example Rathore et al., 2024) report that the combination outperforms either method alone because the PINN loss is ill-conditioned.

9.9 How it is implemented in this code

1. After the Adam epochs, `train_lbfgs()` sets the physics weight to 1 and builds the fixed grid of 4,000 times.
2. All 12,803 weights are flattened into one long list of numbers, because SciPy works with flat vectors.
3. A function `fun(flat)` writes the list back into the network, computes the total loss and its gradient with TensorFlow (compiled with `tf.function` for speed), and returns both to SciPy.

4 SciPy runs L-BFGS-B. After each step, a callback records the loss terms into `loss_history`, continuing the epoch count after the Adam epochs. That is the green stretch in the loss plots.

5 When SciPy stops, the best weights it found are written back into the network.

9.10 What it achieved here

	After Adam (10,000 epochs)	After L-BFGS (13,757 steps)	Improvement
Total training loss	1.1×10^{-5}	6.5×10^{-9}	about 1,700×
Relative L2 error (worst of x, y, z)	6.5×10^{-5} *	1.4×10^{-6}	about 47×
Worst single-point error	2.8×10^{-3} *	5.5×10^{-5}	about 50×
Time	about 3 minutes	about 16 minutes	

* From the separate Adam-only run with the same settings. Across experiments, adding 5,000 L-BFGS steps improved every setting by roughly 15× to 1,000×, and even rescued plain SGD from 23% error to 1.1×10^{-5}.

9.11 Strengths and weaknesses

Strengths	Weaknesses
Uses curvature, so it converges far faster than first-order methods near a minimum.	Each step is more expensive (several loss evaluations in the line search).
No learning rate to tune: the line search chooses the step length.	Needs a fixed, deterministic loss; does not work well with random mini-batches.
Small memory: only m pairs of vectors.	Local: needs a good starting point, otherwise it can stall.
Very mature, reliable implementations (SciPy's dates back to 1990s Fortran code).	Runs on the CPU via SciPy here, with weights copied back and forth every evaluation.
Monotone: accepted steps always lower the loss.	Can stop on evaluation limits (maxfun) before reaching the step limit, as happened here.

9.12 Is L-BFGS “gradient descent”? Is the model still a PINN?

L-BFGS is **gradient-based** (it only ever computes gradients) but it is not plain gradient descent: it reshapes the gradient using estimated curvature and chooses its own step length. It belongs to the **quasi-Newton** family. If a requirement says strictly “gradient descent,” the Adam-only or SGD-only results are the ones that qualify.

The model is still a PINN either way. What makes it physics-informed is the **loss** (data + physics residual), and every optimizer minimizes exactly that same loss. The original PINN paper itself used L-BFGS.

10. What we tried and what we found

In plain words

We ran 49 training experiments in four groups. Each changes one or two settings and measures the error against the RK4 truth at all 10,001 time points. All runs used dt = 0.0001, T = 1, 200 data points and the same network size.

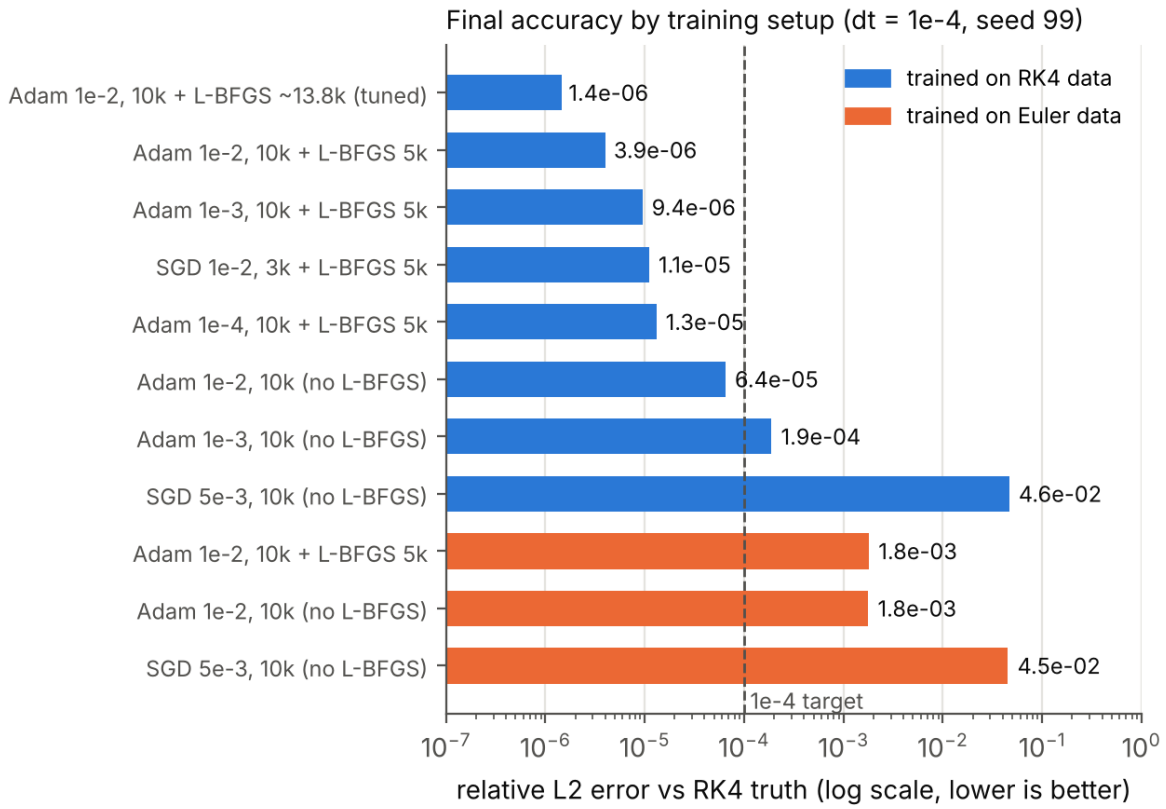


Figure 2. Final relative L2 error for representative setups. Blue: trained on RK4 data. Orange: trained on Euler data. The dashed line is the 10^{-4} target.

Experiment 1: how to split time between Adam and L-BFGS (Adam at 0.001, two seeds)

Adam epochs	L-BFGS steps	Relative L2 (seed 99 / 7)	Worst point (seed 99 / 7)	Time
10,000	2,000	1.6e-5 / 2.2e-5	8.5e-4 / 1.4e-3	about 10 min
5,000	5,000	1.3e-5 / 1.1e-5	4.3e-4 / 4.7e-4	about 13 min
3,000	5,000	8.1e-6 / 1.1e-5	4.4e-4 / 3.3e-4	about 12 min
5,000	10,000	5.2e-6 / 3.8e-6	1.6e-4 / 1.5e-4	about 24 min

Finding: Adam past about 3,000 epochs (at this learning rate) adds little; L-BFGS steps are where accuracy comes from. The two seeds differ by up to about 1.5×, so small differences are noise.

Experiment 2: optimizer and learning rate (seed 99)

Covered in Section 8. Summary: Adam at 0.01 was best; SGD worked only in a narrow band. With 5,000 L-BFGS steps added, all reached 4×10^{-6} to 1.3×10^{-5}: Adam 0.01 3.9e-6, Adam 0.001 9.4e-6, SGD 0.01 1.1e-5, Adam 0.0001 1.3e-5.

Experiment 3: Euler data vs RK4 data

Setup (dt = 0.0001)	Trained on Euler	Trained on RK4
Adam 0.01, 10k + L-BFGS 5k	1.8×10^{-3}	3.9×10^{-6}
Adam 0.001, 10k + L-BFGS 5k	1.8×10^{-3}	9.4×10^{-6}
Adam 0.0001, 10k + L-BFGS 5k	1.8×10^{-3}	1.3×10^{-5}
SGD 0.01, 3k + L-BFGS 5k	1.8×10^{-3}	1.1×10^{-5}
Adam 0.01, 10k, no L-BFGS	1.8×10^{-3}	6.5×10^{-5}
Adam 0.001, 3k, no L-BFGS	2.0×10^{-2}	2.2×10^{-2}
SGD 0.005, 10k, no L-BFGS	4.5×10^{-2}	4.6×10^{-2}

- When training is weak (errors around 10^{-2}), the data source barely matters: the network's own mistakes are far bigger than Euler's.
- Once training is good, Euler becomes a wall: every Euler-trained run stops at about 1.8×10^{-3}, whatever the optimizer.
- Interestingly, the PINN ends up *between* Euler and the truth: 1.8×10^{-3} from the truth (better than Euler's own 3.0×10^{-3}) yet 1.3×10^{-3} from Euler. The physics loss pulls it toward the real equations while the data loss pulls it toward Euler's mistakes.

Experiment 4: the tuned model

Setting	Value
Reference	RK4, dt = 0.0001
Stage 1	Adam, learning rate 0.01 with cosine decay, 10,000 epochs
Stage 2	L-BFGS, 13,757 steps (stopped by the 15,000-evaluation limit)
Relative L2 error	x 1.37e-6, y 1.20e-6, z 7.39e-7; combined 8.9e-7
Worst single-point error	5.5e-5
Largest 3D distance error	6.3e-5
Training time	about 19 minutes on 4 CPU cores

Key takeaway

The tuned model is the only configuration that meets 10^{-4} on every measure, including the strict worst-single-point and 3D-distance errors.

11. Reading the graphs

In plain words

All graphs below come from the tuned model and are produced by the `Lorenz_Visuals` class in `lorenz_pinn_full.py` (method `plot_all`). A **log scale** appears on several of them: each grid line is 10 times the one below it, so very large and very small numbers fit on one picture. Moving up one line means “10 times bigger.”

Reference trajectory in 3D (reference_3d.png)

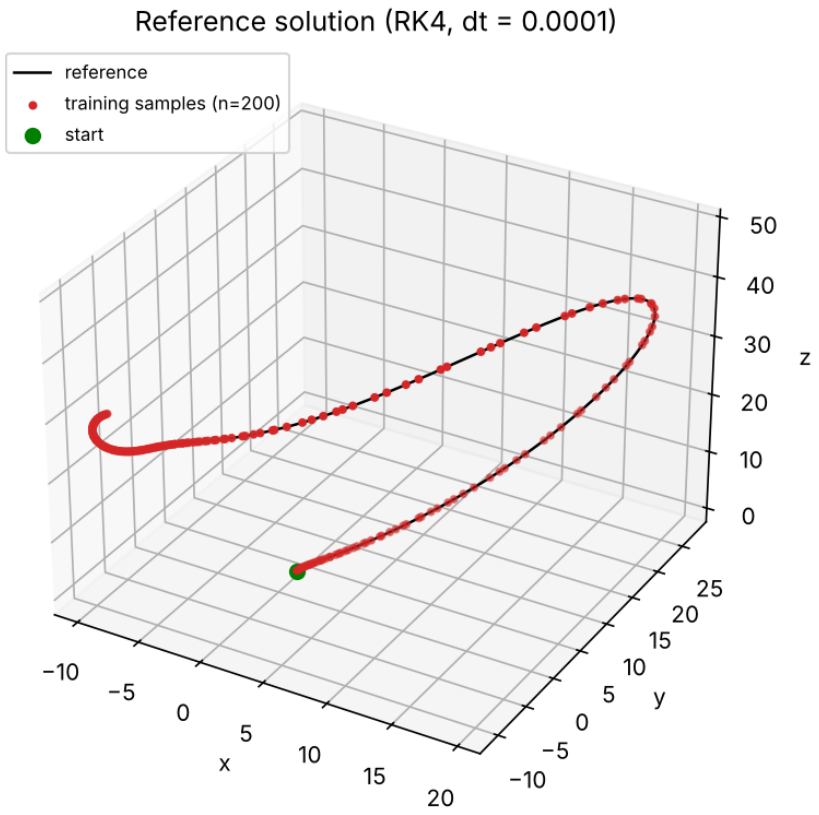


Figure 3. The RK4 path in (x, y, z) space from t = 0 to 1, with the 200 training points marked.

This is the “truth.” The path leaves (1, 1, 1), swings up to about z = 48, and starts curling into one wing of the butterfly. Red dots are the 200 points the network was shown.

PINN prediction in 3D (predicted_3d.png)

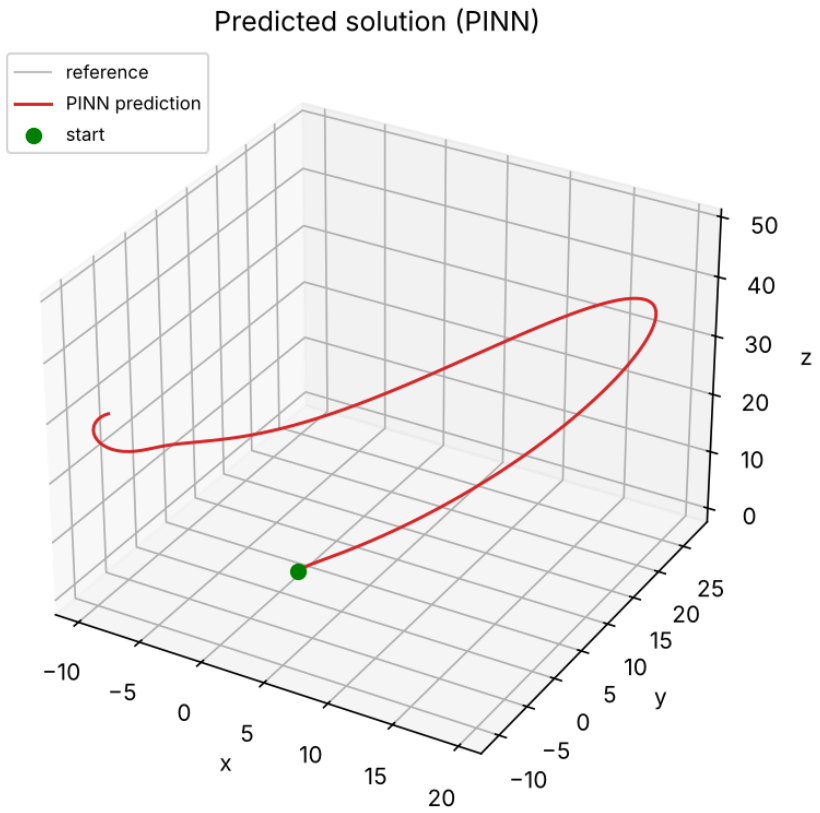


Figure 4. The network's predicted path (red) drawn over the reference (gray). The green dot marks the start.

The gray reference is hidden underneath the red prediction because the two overlap to within about 0.00006, far thinner than the line width.

Each variable over time (time_series.png)

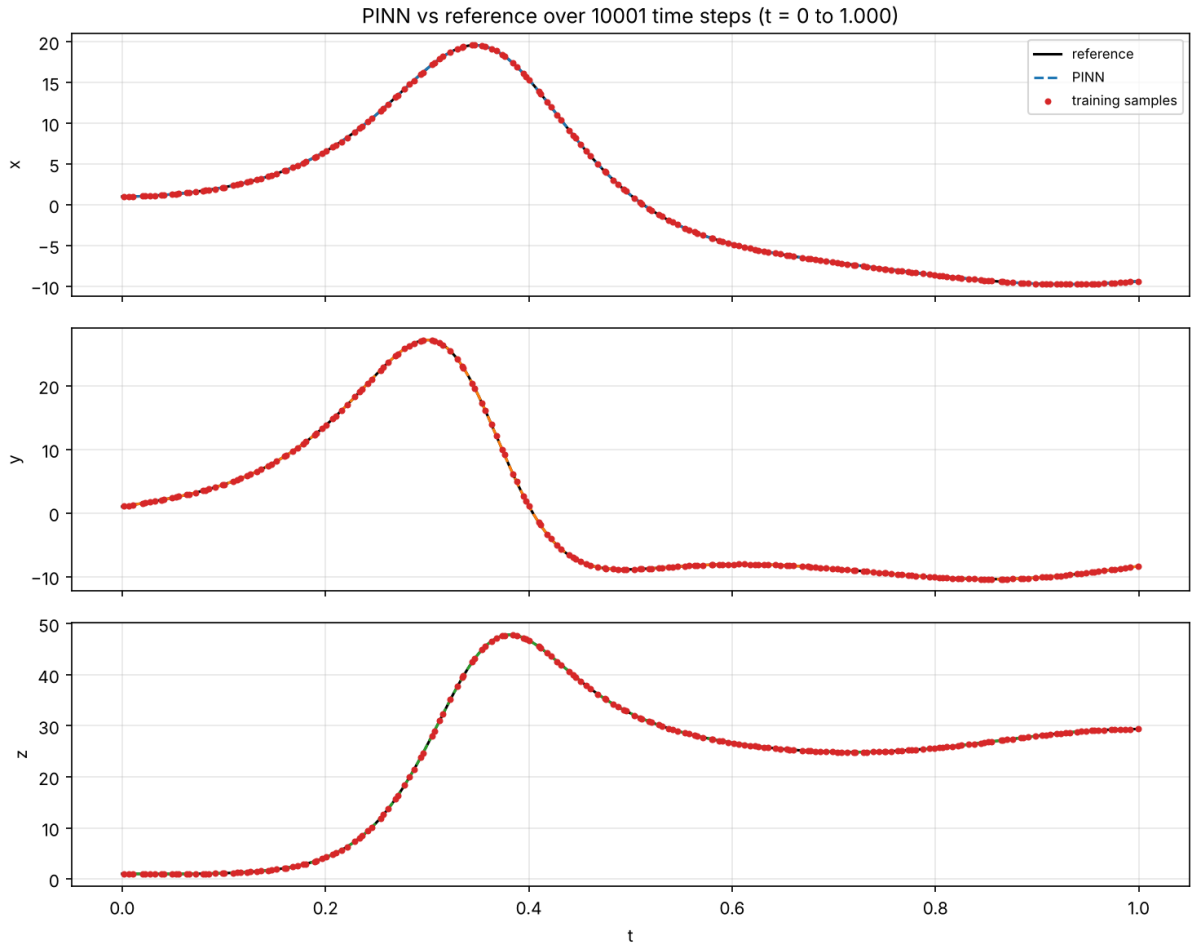


Figure 5. x, y and z against time. Black: reference. Colored dashes: PINN. Red dots: training points.

Each panel shows one variable. x and y peak around t = 0.3, z peaks near t = 0.38 at about 48, then all three settle into slower motion. The PINN line sits exactly on top of the reference everywhere, including between the red training dots, which is where the physics loss does its work.

Error at every time point (errors.png)

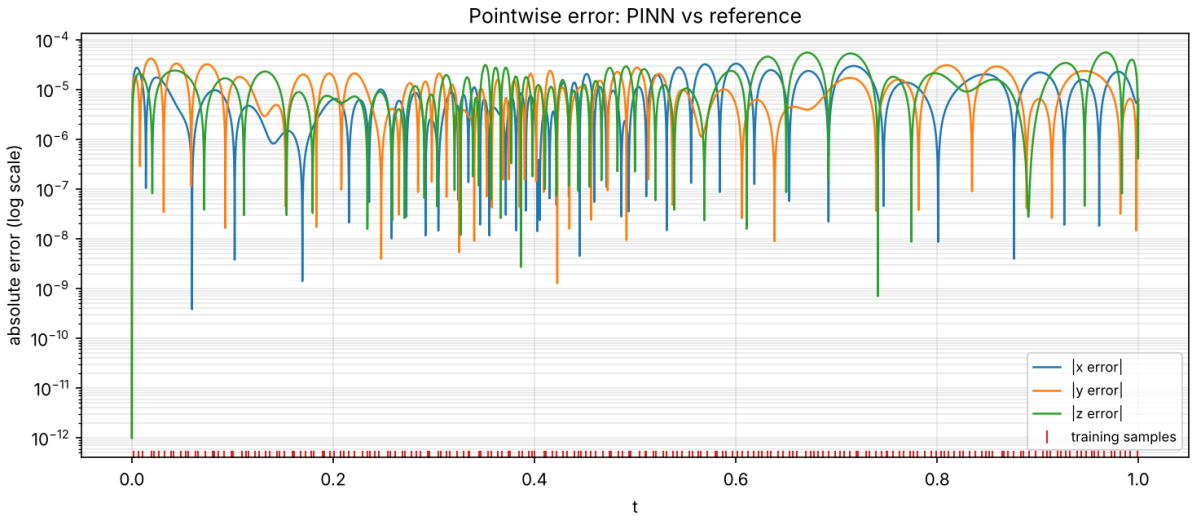


Figure 6. Absolute error |PINN – reference| for x, y and z at all 10,001 times, on a log scale. Red ticks along the bottom mark the training points.

The error stays between about 10^{-9} and 6×10^{-5} across the whole interval, never crossing 10^{-4}. At t = 0 it is essentially zero because the starting point is built in. The sharp downward spikes are moments where the prediction crosses the true value (the error passes through zero as it changes sign); on a log scale those dips look dramatic but are harmless. The error is spread fairly evenly, with no region where the network loses track.

Loss terms during training (losses.png)

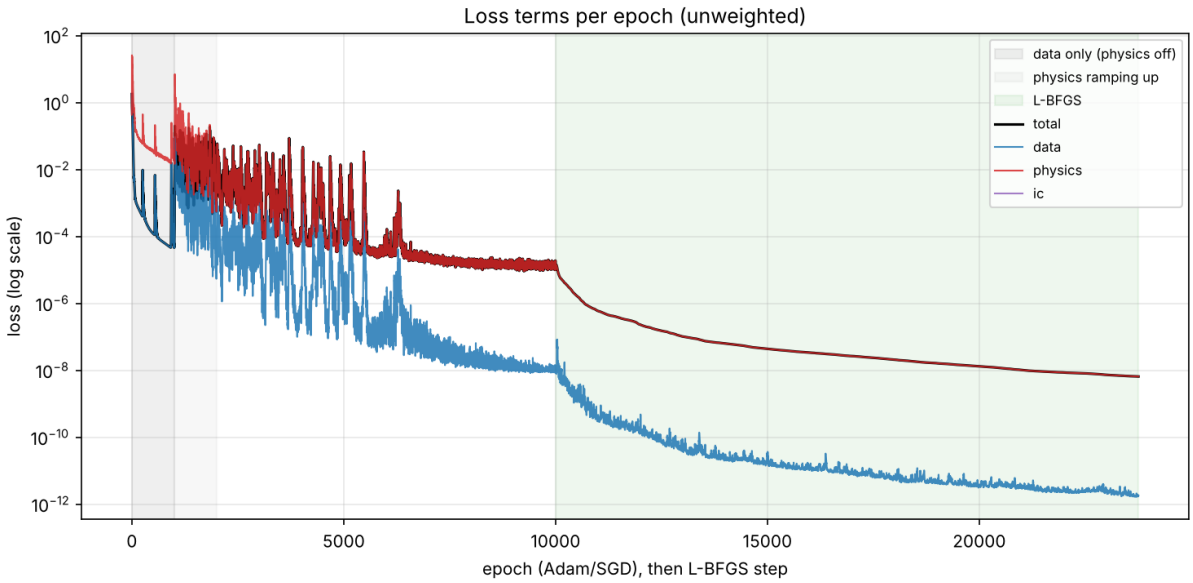


Figure 7. Each loss term over the whole training run, log scale. Gray bands: warm-up. Green: the L-BFGS stage.

- **Dark gray band (0 to 1,000):** data only. The data loss (blue) falls fast; the physics loss (red) is computed but not yet used.

- **Light gray band (1,000 to 2,000):** the physics weight ramps up. The total jumps because a new term is being added, not because training got worse.
- **2,000 to 10,000 (Adam):** both losses fall with lots of jagged spikes. The spikes come from the random physics points (each step checks different times) and from Adam occasionally overshooting. The decaying learning rate calms it down toward 10,000.
- **Green, 10,000 to about 23,750 (L-BFGS):** the curve becomes smooth, because L-BFGS uses a fixed grid and only accepts steps that lower the loss. The physics loss drops from about 10^{-5} to 6×10^{-9}.
- The physics loss (red) is larger than the data loss (blue) throughout: matching 200 points is easy; obeying the equations at thousands of times is the hard part. The total (black) lies on top of the red line for that reason. The IC loss is always 0 and so does not appear.

Best-so-far losses (loss_minima.png)



Figure 8. Losses plotted only at steps where the total loss reached a new lowest value, starting after the warm-up.

Because the raw loss bounces around, this plot keeps only the steps that set a new record low, which shows the true downward trend. During Adam, records are rare (a few dozen in 8,000 steps). During L-BFGS almost every step is a record, which is why the green region is a dense line. The title reports how many steps set records out of the total.

Animation (prediction.gif)

The file `prediction.gif` (in the results folder) animates the PINN drawing the path over time against the reference. A PDF cannot play animations, so it is provided separately.

12. Conclusions, limits and next steps

What we learned

1. **The answer key matters most.** With Euler data, no learning setting got past about 0.2% error. With RK4 data, the same settings reached 0.0001%.
2. **Adam beats plain SGD** by about 1,000× at the same step count and tolerates a much wider range of learning rates.
3. **A higher learning rate with decay (0.01 → 0.0001) worked best** for Adam.
4. **L-BFGS is what reaches the last digits**, improving the loss about 1,700× and the error about 50× after Adam.
5. **Building in the starting point** removes one loss term and makes t = 0 exact.
6. **The tuned PINN meets 10^{-4} on every measure**: relative 1.4×10^{-6}, worst point 5.5×10^{-5}, 3D distance 6.3×10^{-5}.

Limits of these results

- Each setting was mostly run with one random seed. A second seed in Experiment 1 moved results by up to 1.5×, so small differences should not be over-interpreted.
- Only T = 1 was studied. Longer windows are much harder because of chaos.
- The worst-point margin is real but modest (5.5×10^{-5} against a 10^{-4} limit).
- Run times were measured on a 4-core cloud CPU with no GPU.

Possible next steps

- Repeat the tuned run with several seeds and report the average and spread.
- Raise SciPy's `maxfun` so L-BFGS can use all 20,000 steps; the loss was still falling when it stopped.
- Try longer windows (T = 2, 5) with time-marching: train on [0, 1], then use the end as the start of [1, 2], and so on.
- Try the inverse problem: start with wrong σ, ρ, β and let the PINN recover them.
- Run on a GPU to cut training time.

13. Glossary

Term	Meaning
Activation function (tanh)	The smooth squashing function inside each neuron; tanh maps any number into –1 to 1.
Adam	An optimizer: gradient descent with momentum and a separate step size per weight.
Automatic differentiation	Software technique that computes exact derivatives of a program's output.
Chaos	Extreme sensitivity to starting conditions; tiny differences grow exponentially.
Collocation points	Times where the physics equations are checked during training.
Cosine decay	A learning-rate schedule that shrinks the rate along a smooth half-cosine curve.
Derivative (dx/dt)	Rate of change: how much x changes per tiny bit of time.
dt	Time step size of a step-by-step solver.
Epoch	Here, one training step (one loss evaluation and one weight update).
Euler's method	Simplest step-by-step solver; first-order accurate (error proportional to dt).
float32 / float64	Ways of storing numbers with about 7 or 16 significant digits.
Gradient	The slope of the loss with respect to every weight; points uphill.
Gradient descent	Repeatedly stepping the weights opposite to the gradient.
Hessian	Matrix of second derivatives; describes the curvature of the loss.
Hyperparameter	A setting chosen before training (learning rate, epochs, network size).
Ill-conditioned	A landscape steep in some directions and flat in others; hard for simple methods.
Initial condition (IC)	The starting values, here x = y = z = 1 at t = 0.
L-BFGS	Limited-memory quasi-Newton optimizer that estimates curvature from recent gradients.
Learning rate	Size of each weight update.
Line search	Trying step lengths along a direction until the loss drops enough.
Loss	Single score of how wrong the network is during training; the thing being minimized.
Lorenz system	Three coupled equations from a 1963 weather model; a classic example of chaos.
Neural network	A flexible formula with many adjustable weights, trained from examples.
Optimizer	The procedure that adjusts weights to lower the loss (SGD, Adam, L-BFGS).
PINN	Physics-informed neural network: trained on data and on obeying known equations.
Quasi-Newton	Optimizers that approximate Newton's method without computing the Hessian.
Relative L2 error	Size of the error divided by size of the true values, using root-sum-of-squares.
Residual	How far the network's rate of change is from what the equations require.
RK4	Fourth-order Runge–Kutta solver; error proportional to dt^4.
Seed	Number that fixes the random choices so a run can be repeated exactly.
SGD	Stochastic gradient descent: gradient descent with a randomly sampled gradient.
Weights	The adjustable numbers inside a neural network.

14. How to reproduce the results

1. Install Python 3 with `numpy pandas tensorflow scipy matplotlib`.
2. Run `python scripts/lorenz_pinn_full.py`. It trains the tuned model (about 20 minutes on a laptop-class CPU) and saves every graph to `./plots`.
3. To compare settings, create the solver with different arguments, for example `Lorenz_PINN_solver(optimizer="sgd", learning_rate=5e-3, lbfgs_iters=0)` or `method="euler"`.
4. After `pinn_pipeline()`, `solver.pred_df` holds the reference, prediction and error columns (x_err, y_err, z_err, err_3d) at every time point, and `solver.loss_df` holds the loss history.

All experiment results are in `results/all_results.csv` in the project files.

15. Validation: training on part of the time window

In plain words

Up to now the network saw sample points from the whole window, t = 0 to 1, and was graded on that same window. That shows it can **fit** the path, but not whether it can **predict** parts it never saw. Here we hide part of the window: the network only gets data points from one or more chosen pieces of time (the **training intervals**), and we grade it on everything else (the **held-out** time).

We tried this two ways. In the first, the physics rules are also only checked inside the training pieces, so outside them the network knows nothing but the starting point. In the second, the rules are checked on the whole window and only the data is hidden. That is the realistic PINN case: the equations are known everywhere, measurements are not.

Short answer: with the physics checked everywhere, 27 of 35 layouts predicted the hidden time to 10^{-5} or better (relative L2, mostly 2 to 9 × 10^{-6}), even with only 5% of the window as data. Rerun with the full tuned settings, that 5% layout meets 10^{-4} on both measures (1.0 × 10^{-6} average, 6.4 × 10^{-5} worst point). With the physics checked only where the data is, the network cannot see past the last piece, and only closely spaced pieces covering half the window came under 10^{-4}.

What was added to the code

All of it lives in a new file, `scripts/interval_validation.py`, next to `lorenz_pinn_full.py`. Every run uses the tuned settings from Section 10 (RK4 data at dt = 10^{-4}, float64, Adam at 0.01 for 10,000 epochs, then L-BFGS, 200 data points per unit of time). Nothing about the network was re-tuned.

Piece	What it does
Lorenz_Interval_Validation	A subclass of Lorenz_PINN_solver. Takes *train_intervals*, e.g. [(0, 0.3), (0.6, 0.8)], and *physics_on* = "train" or "full". Samples data only inside the intervals and places the physics points accordingly; the network and training are inherited unchanged.
make_intervals, parse_intervals	Build interval lists: n equal pieces with a given total length, spread end to end, centered, or at random; or parse text like "0-0.3, 0.6-0.8".
Interval_Validation_Runs	Keeps every run as a row in a pandas DataFrame (*results_df*) and rewrites *results.csv* after each run. Runs already in the CSV are skipped, so a stopped sweep picks up where it left off.
Interval_Validation_Visuals	Plots for one run (error over time, x/y/z over time and the 3D path, with training intervals shaded) and for many runs (the heatmaps and line charts in this section).
Command line	--intervals, --n-intervals with --total-length and --placement, --physics-on, --sweep single|multi|budget, --plot results.csv. See the end of this section.

Two small changes were needed in the base solver: the code that picks physics points is now in two methods (`random_colloc`, `fixed_colloc`) so the subclass can move them. The tuned model's results are unchanged.

One fix on the way. The network's outputs are scaled by the mean and spread of x, y and z. At first those came from the training points only. On a short interval the spread is tiny, the scaled physics error becomes enormous and Adam stalled (a data loss of 37 on [0.5, 0.75], when it should be near 10^{-10}). The validation runs now use the same three scaling numbers as the tuned model, taken from the reference path. These are just units, like choosing meters over millimeters; they carry no detail about the path. The old behavior is still available as `scale_from="train"`.

How the runs are scored

Every run is compared with the RK4 reference at all 10,001 time points, split into regions:

- **train**: inside the training intervals.
- **held out**: everything else. This is the number that matters here.
- **gaps**: held-out time between the first and last interval (filling in).
- **beyond**: held-out time before the first or after the last interval (predicting forward or back).

Each region gets the same two scores as before: relative L2 (overall size of the error) and the worst single-point error. Sweeps used 5,000 L-BFGS steps instead of 20,000 to keep each run to about 1 to 17 minutes; the best layouts were then rerun with the full tuned budget (below).

Experiment A: one interval, moving it and stretching it

One training interval, with length 0.1, 0.25, 0.5 or 0.75 and start time 0, 0.25, 0.5 or 0.75 (13 layouts that fit in [0, 1]), each run both ways: 26 runs.

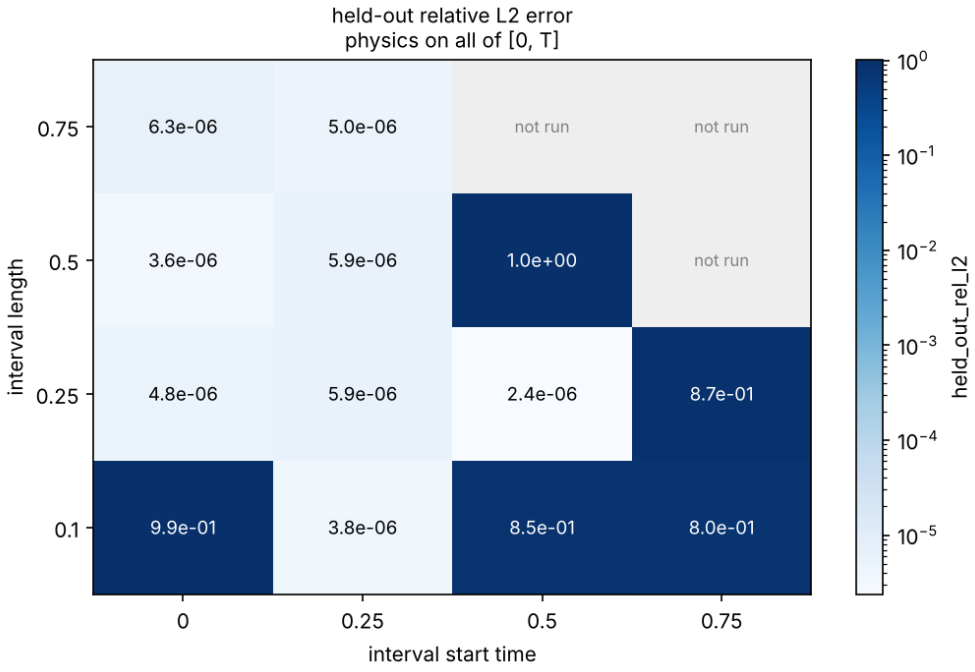


Figure 15.1. Held-out relative L2 error, physics on the whole window. Rows: interval length. Columns: start time. Light = small error. Gray cells don't fit inside [0, 1].

With the physics checked everywhere, 8 of 13 layouts predict the hidden time to 2 to 6 × 10^{-6}, the same level as the tuned model on its own training window. The 5 that fail do so completely (around 100% error, and they don't even fit their own data): [0, 0.1], [0.5, 0.6], [0.5, 1], [0.75, 0.85] and [0.75, 1]. Data has to sit somewhere in the middle of the window. With data only right at the start or only late, the physics on the long empty stretch pulls the network toward a smooth wrong path before the data can correct it. This is a known weakness of PINNs on chaotic systems. One success, [0.5, 0.75], turned out to be borderline: it worked from only 1 of 4 starting points (see the starting-weights check below).

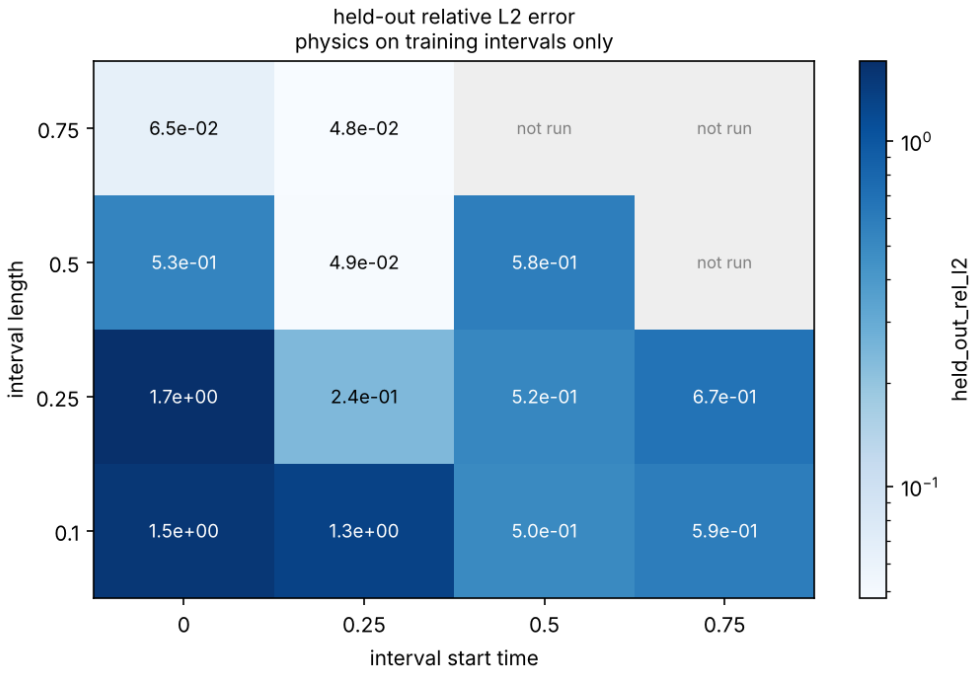


Figure 15.2. The same grid with physics only on the training interval.

With the physics only inside the interval, nothing works: held-out errors run from 5% to over 100%. The network fits its interval very well (train error about 10^{-6}), but outside it there is no information at all, and a neural network does not extrapolate a chaotic path by itself. Longer intervals do better simply because less time is hidden.

Experiment B: several intervals

The same total length (0.25 or 0.5) broken into 2, 3 or 5 equal pieces, either spread from end to end of [0, 1] or packed into the first part of the window: 24 runs.

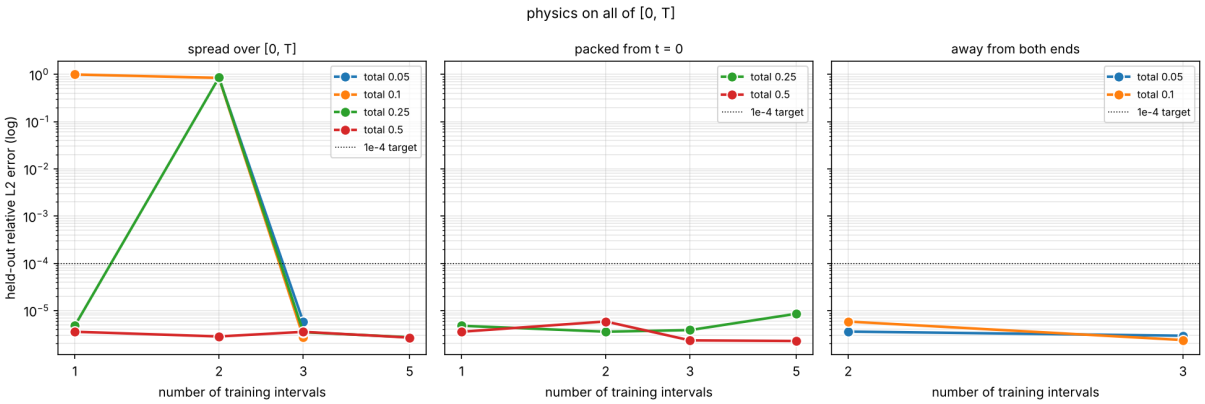


Figure 15.3. Held-out relative L2 error against the number of pieces, physics on the whole window. Panels: how the pieces are placed. Colors: total training length (includes Experiment C's 0.05 and 0.1 runs).

With the physics everywhere, the number of pieces barely matters: all but one layout land at 2 to 9 × 10^{-6}. The failure is two short pieces at the very ends ([0, 0.125] and [0.875, 1]; the same shape also fails at 0.05 and 0.1 total), which leaves the whole middle without data, matching Experiment A. With longer end pieces ([0, 0.25] and [0.75, 1]) it works.

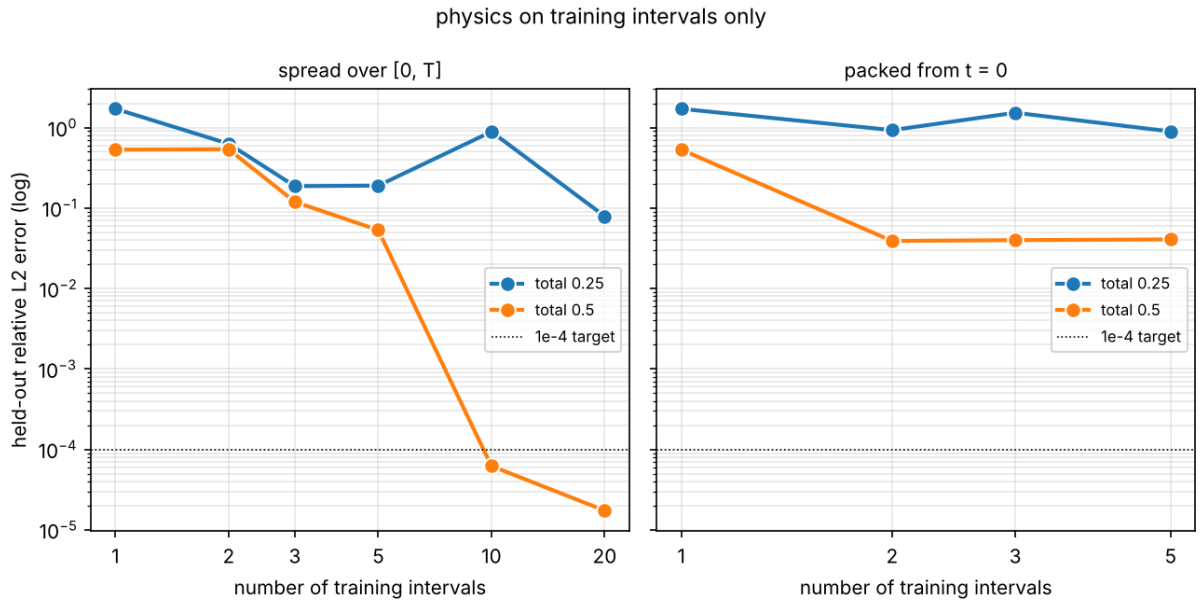


Figure 15.4. The same with physics only on the training pieces (includes Experiment C's 10- and 20-piece runs).

With physics only on the pieces, splitting does help, because the error in the **gaps** between pieces is small when the gaps are short (1 to 6 × 10^{-5} for gaps up to about 0.06, then growing fast: 2 × 10^{-4} to 5 × 10^{-3} at 0.06 to 0.125). What still fails is the time **beyond** the last piece. Packed layouts keep 5% to over 100% error there. Only layouts that reach both ends with short gaps get under 10^{-4}.

Experiment C: how little data is enough?

The ideal is to choose length, location and number together, with total length well under the window. Experiments A and B narrow the search: with the physics everywhere, location matters most and

length hardly at all. So this round shrinks the total to 0.05 and 0.1 (5% and 10% of the window) and tries one piece at t = 0.3, and 2 or 3 pieces either spread end to end or centered away from the ends. For physics only on the data, it tests 10 or 20 pieces covering the window.

Physics on	Total	Pieces	Intervals	Held-out rel. L2	Held-out worst point
all of [0, 1]	0.05	3	[0.237, 0.254], [0.492, 0.508], [0.746, 0.762]	2.9×10^{-6}	2.7×10^{-4}
all of [0, 1]	0.05	2	[0.317, 0.342], [0.658, 0.683]	3.6×10^{-6}	2.7×10^{-4}
all of [0, 1]	0.05	3	[0, 0.0167], [0.492, 0.508], [0.983, 1]	5.7×10^{-6}	3.4×10^{-4}
all of [0, 1]	0.05	1	[0.275, 0.325]	1.5×10^{-5}	1.1×10^{-3}
all of [0, 1]	0.05	2	[0, 0.025], [0.975, 1]	8.4×10^{-1}	4.6×10^{1}
all of [0, 1]	0.1	3	[0.225, 0.258], [0.483, 0.517], [0.742, 0.775]	2.4×10^{-6}	1.6×10^{-4}
all of [0, 1]	0.1	3	[0, 0.0333], [0.483, 0.517], [0.967, 1]	2.8×10^{-6}	1.9×10^{-4}
all of [0, 1]	0.1	2	[0.3, 0.35], [0.65, 0.7]	5.9×10^{-6}	4.1×10^{-4}
all of [0, 1]	0.1	1	[0.25, 0.35]	1.1×10^{-5}	7.1×10^{-4}
all of [0, 1]	0.1	2	[0, 0.05], [0.95, 1]	8.4×10^{-1}	4.6×10^{1}
pieces only	0.25	20	20 pieces spread over [0, 1]	7.9×10^{-2}	1.7×10^{1}
pieces only	0.25	10	10 pieces spread over [0, 1]	8.9×10^{-1}	1.2×10^{2}
pieces only	0.5	20	20 pieces spread over [0, 1]	1.7×10^{-5}	1.1×10^{-3}
pieces only	0.5	10	10 pieces spread over [0, 1]	6.2×10^{-5}	2.9×10^{-3}

Finding. Three pieces of 0.017 each (5% of the window) placed at about 0.25, 0.5 and 0.75 predict the other 95% to 3×10^{-6} relative L2. The same 5% as one piece at t = 0.3 is 5× worse on average and 4× worse at its worst point. Spreading a little data through the middle beats concentrating it. With physics only on the data, 10 or 20 pieces covering half the window get to 2 to 6×10^{-5}; with a quarter of the window covered they fail.

Final check with the full tuned settings

The sweeps stopped L-BFGS at 5,000 steps, and in them the worst-point error sat a bit above 10^{-4} (mostly 1 to 5×10^{-4}), even when the average was 50× under the target. Four of the best layouts were rerun with the tuned budget of 20,000 L-BFGS steps:

Layout	Physics on	Data	Held-out rel. L2	Held-out worst point	Meets 10^{-4}?	L-BFGS steps
[0.225, 0.258], [0.483, 0.517], [0.742, 0.775]	all of [0, 1]	0.1	8.9×10^{-7}	4.9×10^{-5}	both	13816
[0.237, 0.254], [0.492, 0.508], [0.746, 0.762]	all of [0, 1]	0.05	1.0×10^{-6}	6.4×10^{-5}	both	13837
[0.5, 0.75]	all of [0, 1]	0.25	8.5×10^{-1}	4.7×10^{1}	no	14084
20 pieces over [0, 1]	pieces only	0.5	5.7×10^{-6}	4.5×10^{-4}	rel. L2 only	13771

Both layouts with 5% and 10% data now meet 10^{-4} on both measures, like the tuned model trained on the full window. With 5%, three pieces of 0.017 each, the network never saw 95% of the path and still gets it to 1.0×10^{-6} on average, with a worst point of 6.4×10^{-5}. The 20,000 extra L-BFGS steps took the worst point from 1.6 to 2.7×10^{-4} down to 4.9 to 6.4×10^{-5}. As in Section 10, scipy stopped at its 15,000-evaluation limit (about 13,800 steps). The 20-piece layout with physics only on the data reached 5.7×10^{-6} on average but 4.5×10^{-4} at its worst. [0.5, 0.75] failed on this rerun even though it had worked in the sweep; the next subsection explains why.

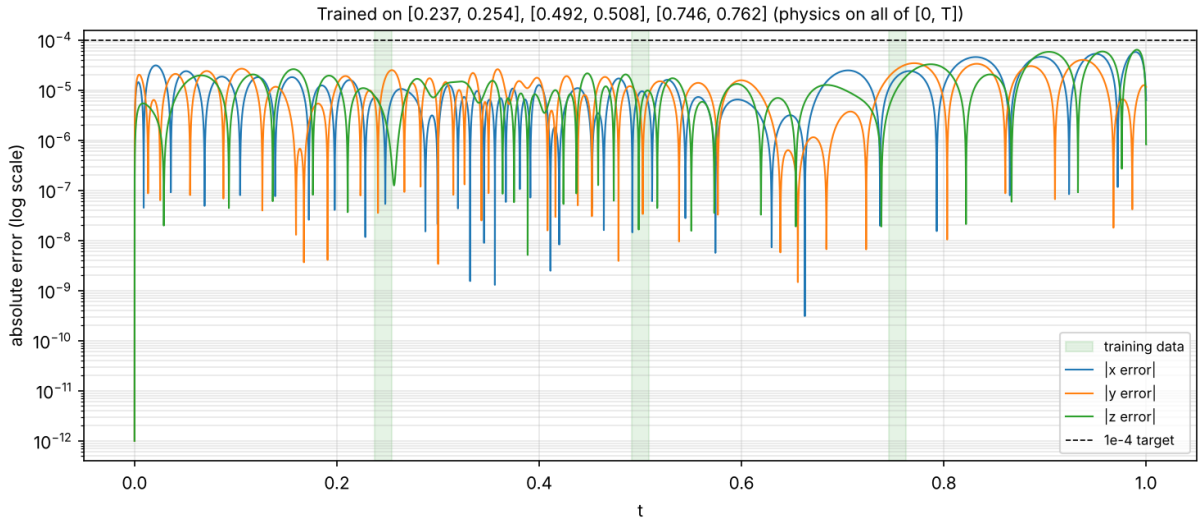


Figure 15.5. Error over time for the 5%-data layout with the full tuned settings, at all 10,001 times. Green bands: the only times the network saw data. Dashed line: 10^{-4}.

Does it depend on the starting weights?

The network starts from random weights, and the seed decides which ones. While checking the reruns we found that the seed did not fully pin them: Keras's weight initializer also depended on how many models the program had built before. So each sweep run above started from effectively different random weights, which is why [0.5, 0.75] worked in the sweep but failed when rerun alone. The validation class now pins every random source from the seed (one line, `tf.keras.utils.set_random_seed`). Four layouts were then run from two more seeds (5,000 L-BFGS steps):

Layout (physics on all of [0, 1])	Sweep run	Seed 1	Seed 2	Full tuned rerun	Works
3 pieces, 5% total, centered	2.9×10^{-6}	2.8×10^{-6}	3.9×10^{-6}	1.0×10^{-6}	4 of 4
[0.25, 0.5]	5.9×10^{-6}	5.2×10^{-6}	4.0×10^{-6}	n/a	3 of 3
[0.5, 0.75]	2.4×10^{-6}	8.5×10^{-1}	7.6×10^{-1}	8.5×10^{-1}	1 of 4
[0, 0.1]	9.9×10^{-1}	9.9×10^{-1}	9.9×10^{-1}	n/a	0 of 3

Held-out relative L2 error (n/a: not rerun). The clear cases stay clear: the 5% layout and [0.25, 0.5] work every time, and [0, 0.1] fails every time. [0.5, 0.75] is on the edge and mostly fails. So the working/failing map in Experiment A is right in its broad shape, but a single run near the edge can go either way.

What this means

- **The physics does the predicting.** Once the equations are checked on the whole window, a PINN needs very little data, as long as some of it is in the middle. Without the equations outside the data, it cannot predict ahead at all.
- **Location beats length and count.** Where the data sits decides success or failure; how much data there is, and how many pieces it comes in, moves the error by less than a factor of 10 (usually under 3) once the location is right.
- **The failures are total, not gradual.** A layout either reaches about 10^{-6} or misses by 100%. Nothing landed in between.

Limits

- Each sweep layout ran once, from starting weights that were not pinned (see above). Small differences between successful layouts are noise, and layouts near the edge can flip.
- Sweeps used 5,000 L-BFGS steps; only the four layouts above got the full 20,000.
- The grid is coarse (4 lengths × 4 start times, 2 to 20 pieces). The boundary between working and failing layouts is only roughly located.
- Held out means "no data", not "no information": with physics on the whole window the equations are still enforced there, as they would be in a real application.

How to run it

From the `scripts/` folder:

```
python interval_validation.py --intervals "0.25-0.5" --physics-on full
python interval_validation.py --n-intervals 3 --total-length 0.05 \
    --placement centered --physics-on full
python interval_validation.py --sweep single --out runs/single
python interval_validation.py --plot runs/single/results.csv --out runs/plots
```

Or from Python:

```
from interval_validation import (Interval_Validation_Runs,
    Interval_Validation_Visuals, make_intervals)
runs = Interval_Validation_Runs("runs/mine")
runs.run("five_percent", make_intervals(3, 0.05, placement="centered"),
         physics_on="full")
runs.results_df          # one row per run, also in runs/mine/results.csv
Interval_Validation_Visuals(solver=runs.last_solver, save_dir="plots").plot_run_all()
```

All results from this section are in `results/interval_validation/`: `sweep_results.csv` (64 sweep runs), `final_results.csv` (the four full runs), `seed_results.csv` (the starting-weights check), and the plots.

16. Prediction horizon: how far out, and how much data

In plain words

Lorenz is chaotic: two paths that start almost together drift apart, roughly 2.5 times further for every unit of time. So we asked how long a stretch of time, T, the network can predict reliably, which parts of that stretch need data, and how little data is enough.

Short answer: one network trained on the whole stretch stops working past about T = 2.5, even with data everywhere. Splitting the stretch into 1-unit windows and training them one after another (**time-marching**) removes that limit: T = 3 and T = 5 come out as accurate as T = 1. After that, what matters is not how long T is but **how long the network has to go without data**. Short pieces of data at least once per time unit kept T = 5 at about 10^{-5} or better, using as little as 2% to 5% of the time. Leaving a whole time unit without data was hit or miss.

16.1 The question

Sections 10 to 15 all used t = 0 to 1. Brian asked three follow-up questions: for what T are the predictions consistent, which part of the data is most important to sample for each T, and how much or how little data each T needs. Every run here uses the tuned settings (RK4 data at dt = 10^{-4}, a 4 × 64 tanh network, Adam at 0.01 for 10,000 epochs, then L-BFGS) with the same density of points per unit of time as the tuned model: 200 data points and 2,000 physics points per unit. To fit the runs into the time available, L-BFGS was capped at 5,000 steps instead of 20,000. At T = 1 that gives 2.5 × 10^{-6} relative error with a worst point of 1.6 × 10^{-4}, so worst-point errors in this section sit a little above the 10^{-4} target even when everything works. The relative L2 error is the main score here.

16.2 One network has a hard limit around T = 2.5

First, the simplest test: give the network data at every point on [0, T] and make T longer. If chaos were the only problem, data everywhere should keep the network on track at any T.

T	Start 99: relative L2	Start 99: worst point	Start 7: relative L2	Training time
1	2.5 × 10^{-6}	1.6 × 10^{-4}	not run	10 min
1.5	3.8 × 10^{-5}	2.2 × 10^{-3}	not run	15 min
2	1.6 × 10^{-4}	9.9 × 10^{-3}	5.9 × 10^{-5}	17 min
2.5	9.7 × 10^{-4}	4.1 × 10^{-2}	7.9 × 10^{-4}	20 min
3	5.2 × 10^{-1}	4.5 × 10^{1}	4.9 × 10^{-1}	23 min

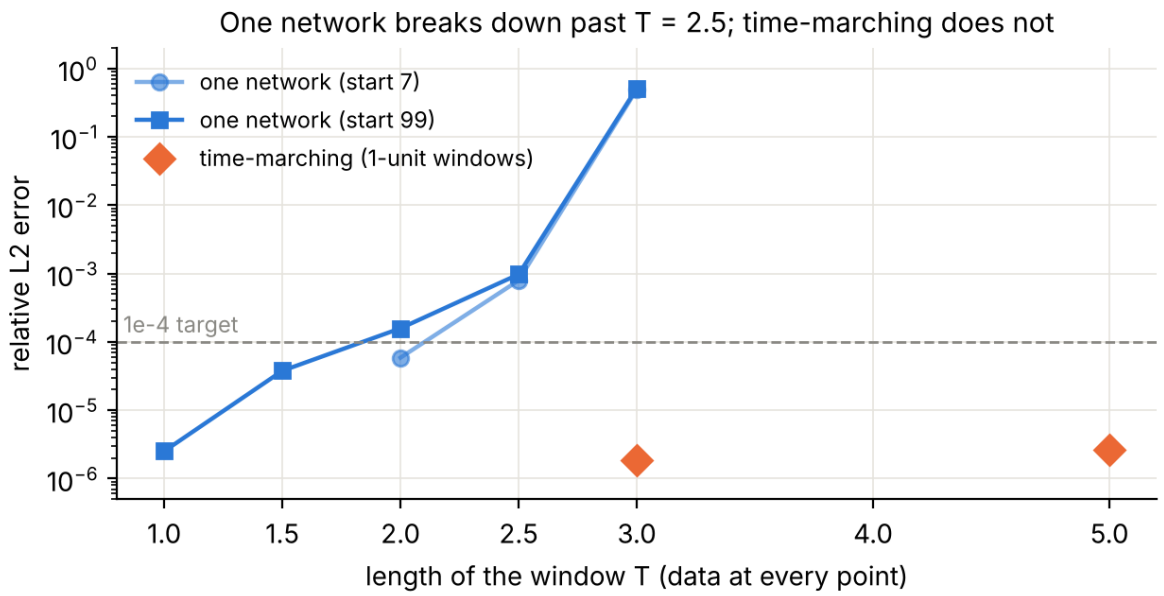


Figure 16.1. Relative error with data at every point. Blue: one network for all of [0, T], from two random starting points. Orange: time-marching (Section 16.3).

The error grows about 10× for every extra half unit of time, and at T = 3 the network fails completely (about 50% error) from both starting points. The training log shows why: the data loss never drops below about 1, so the network does not even fit the 600 points it was shown. This is a **training** failure, not chaos. One small network has to bend through more and more loops of the butterfly as T grows, the physics loss pulls it toward a smooth wrong answer, and the optimizer gets stuck. This is a well-known weakness of PINNs on long time windows.

16.3 Time-marching: one short window at a time

In plain words

Instead of one network for the whole stretch, train a fresh network on [0, 1], read off where it ends at t = 1, use that as the starting point for a second network on [1, 2], and so on. Each network only has to learn one short, simple piece. This is the same idea an ordinary solver like RK4 uses, only with steps of one whole time unit.

The new file `scripts/time_marching.py` does this. `Lorenz_Window` is one window: it reuses the validation solver from Section 15 with the physics checked over the whole window, the starting point built in exactly (hard initial condition) at the previous window's predicted end, and data only where the chosen data pieces fall inside the window. `Lorenz_Time_Marching` runs the windows in order. Each window starts from the previous network's weights, which saves training time, and the results are stitched into one prediction table and scored against one RK4 run over all of [0, T].

T	Windows	Relative L2	Worst point	Training time
3	3	1.8×10^{-6}	1.7×10^{-4}	30 min
5	5	2.6×10^{-6}	2.3×10^{-4}	50 min

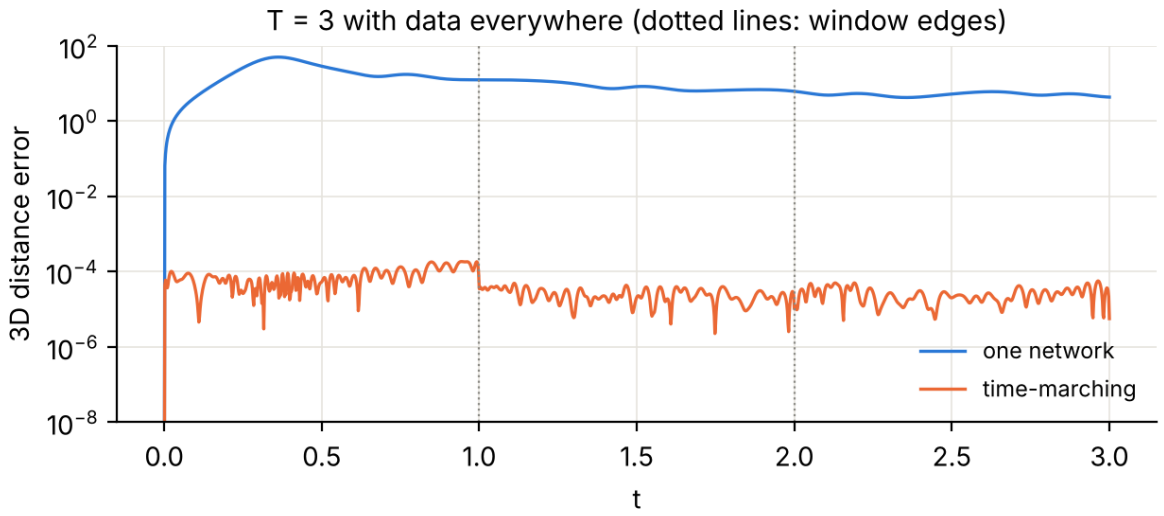


Figure 16.2. Error over time at T = 3 with data everywhere. One network is off by about 10 from the start; time-marching stays near 10^{-4} to 10^{-5} the whole way.

With data everywhere, time-marching makes T = 3 and T = 5 as accurate as the original T = 1 model, and the error at the end of each window does not grow from one window to the next. So the T = 2.5 limit belonged to the single network, not to the problem. Every run from here on uses time-marching.

16.4 Where to sample

Next, the data was cut down and placed in different patterns over T = 5, with the physics still checked everywhere. "Held out" means every time point that had no data.

Data pattern	Share of time with data	Starting points	Held-out relative L2	Result
10% of every window (3 short pieces per window)	10%	1	2.5×10^{-6}	passed 1 of 1
5% of every window (3 short pieces per window)	5%	3	1.4×10^{-5}, 2.9×10^{-6}, 2.4×10^{-6}	passed 3 of 3
2% of every window (3 short pieces per window)	2%	1	1.2×10^{-5}	passed 1 of 1
5% of every other window (windows 1, 3, 5)	3%	2	3.7×10^{-1}, 6.4×10^{-5}	passed 1 of 2
All of windows 1, 3, 5; nothing in 2 and 4	60%	1	3.4×10^{-1}	passed 0 of 1
All of [0, 1]; nothing after	20%	1	5.9×10^{-1}	passed 0 of 1

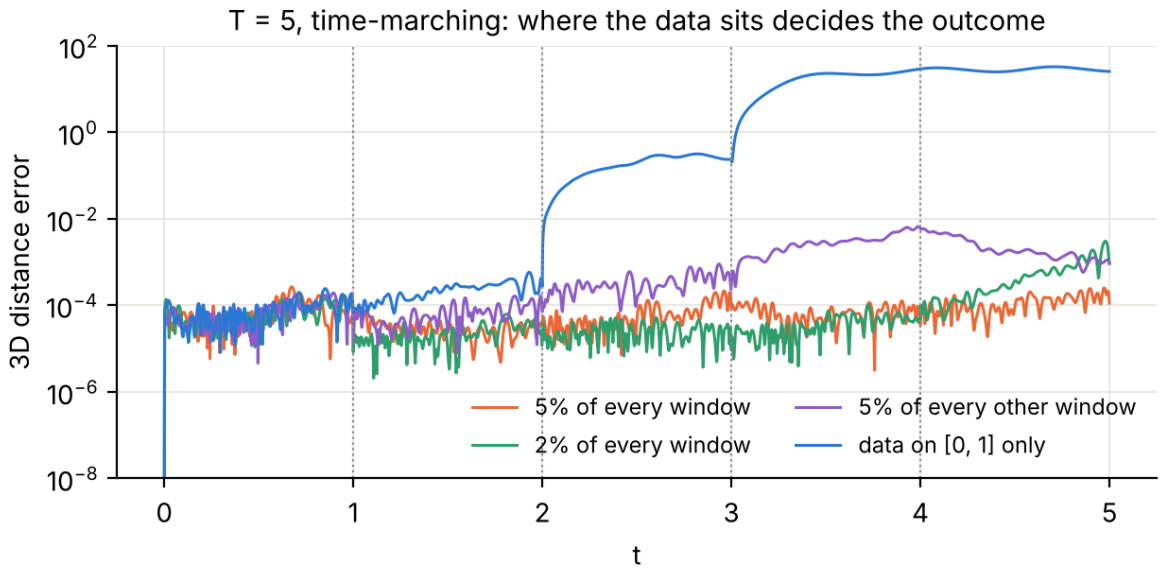


Figure 16.3. Error over time at T = 5 for four data patterns (start 99). Dotted lines mark the window edges.

Three things stand out.

- **Spreading data out beats piling it up.** Three tiny pieces in every window (5% of the time) passed from all three starting points. A full unit of data at the start (20% of the time) failed.
- **Failures are sudden, and they start where a window has no data.** In Figure 16.3 the data-on-[0, 1] run does not drift off slowly the way chaos alone would make it. Its error jumps at t = 2 and again at t = 3, the first moment of a window with no data. With only the physics to go on, a window can settle on a smooth path that obeys the equations but is the wrong path. Chaos alone would only have grown its 3×10^{-4} error at t = 2 to about 10^{-3} by t = 3.
- **One bad window ruins everything after it.** Each window is forced to start exactly where the previous one ended. Once a window lands on the wrong path, the next window starts from a wrong point and even full data cannot pull it back, which is why "all of windows 1, 3, 5" failed despite having data for 60% of the time. Every-other-window layouts passed once and failed once, so they are not reliable.

16.5 How little data: the gap test

Because the failures happen in stretches with no data, the last test asked directly how long the network can go without any. Each data piece is 0.05 long (10 data points), and the pieces are spaced evenly over T = 5.

Time between pieces	First piece at t =	Share of time with data	Held-out relative L2	Held-out worst point	Result
0.5	0.25	10%	1.2×10^{-5}	2.3×10^{-3}	pass
1	0.5	5%	1.0×10^{-5}	8.7×10^{-4}	pass
1.5	0.5	4%	2.3×10^{-3}	2.3×10^{-1}	fail

Time between pieces	First piece at t =	Share of time with data	Held-out relative L2	Held-out worst point	Result
1.5	0.75	3%	5.7×10^{-1}	4.4×10^{1}	fail
2	0.5	3%	3.0×10^{-1}	2.0×10^{1}	fail
2	1	2%	5.4×10^{-1}	4.5×10^{1}	fail

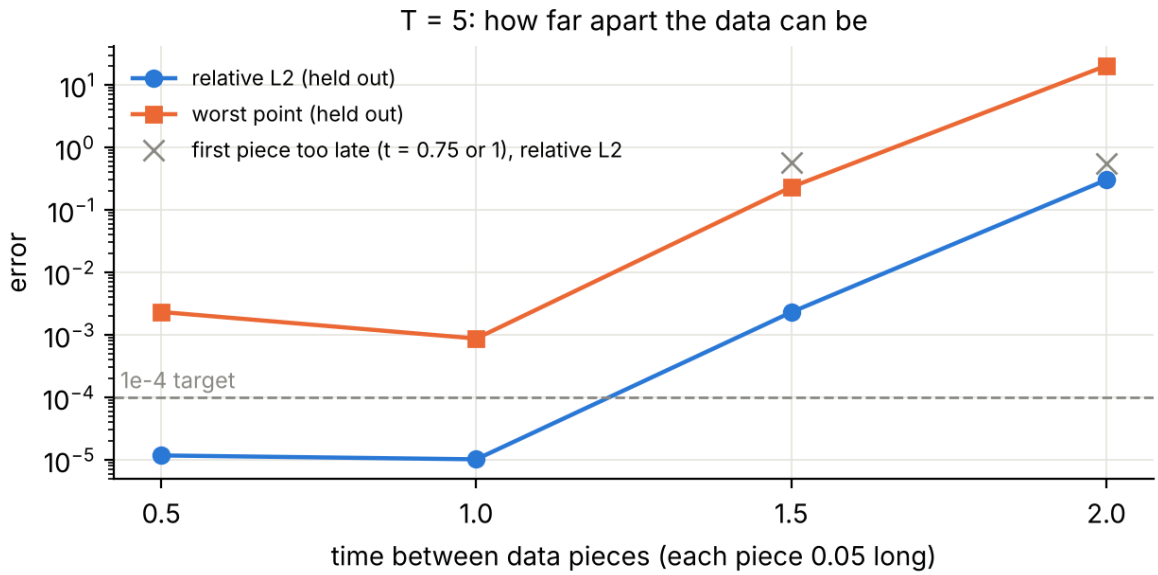


Figure 16.4. Held-out error at T = 5 against the time between data pieces.

Pieces every 0.5 and every 1 time unit both passed, at about 1×10^{-5}. The first attempts at 1.5 and 2 put their first piece at t = 0.75 and t = 1.0, and both failed inside the first window, already off by 10 to 30 at t = 0.5. Section 15 found the same thing on [0, 1]: without data somewhere around t = 0.25 to 0.5, where the path swings up fastest, the network locks onto the wrong path from the start. Rerun with the first piece at t = 0.5, the picture is clearer. **With pieces 1.5 apart**, the prediction stayed near 10^{-4} through three gaps and then broke in the last one, between t = 4 and 5 (relative L2 2.3×10^{-3} overall). **With pieces 2 apart**, it held to 4×10^{-4} through the first gap but broke early in the second, around t = 2.5. So about 1 time unit without data is safe, 1.5 is borderline, and 2 fails. Chaos alone does not explain the failures: errors grow only about 2.5 times per time unit, so a 10^{-4} error would still be under 10^{-3} after 2 units. What goes wrong is the same thing seen in Section 16.4: the longer a window runs on physics alone, the more likely it is to settle on a smooth path that obeys the equations but is the wrong one, and the error then jumps rather than grows.

16.6 The answer for each T

T	Method that works	Least data that worked	Most important to sample
1	One network	5% of the time, 3 pieces (Section 15)	The middle of the window
2	One network (barely), or time-marching	Data everywhere was tested; time-marching rules below apply	At least one piece per time unit
2.5 to 3	Time-marching only	Data everywhere: 1.8×10^{-6} at T = 3	At least one piece per time unit
5	Time-marching only	2% to 5% of the time, as short pieces at most 1 time unit apart	Every time unit; in the first unit, the stretch t = 0.25 to 0.5

Key takeaway

For this model, "how far out can we predict" has two answers. With one network, about T = 2 to 2.5, even with data everywhere. With time-marching, there is no fixed limit up to T = 5, the longest tested, as long as the data never leaves more than about 1 time unit uncovered (1.5 is borderline, 2 fails), and the first unit has data around t = 0.25 to 0.5. The amount of data barely matters (2% to 10% all worked). The spacing decides everything.

16.7 Limits of these results

- Most layouts were run from one random starting point. The 5%-per-window layout was repeated from three and the every-other-window layout from two, and those repeats showed that layouts near the boundary can go either way. Treat single results near the line with care.
- L-BFGS was capped at 5,000 steps to keep each T = 5 run under an hour. Worst-point errors would likely drop with the full 20,000, as they did at T = 1.
- With 1-unit windows, "pieces at most 1 apart" and "data in every window" are almost the same condition, so these runs cannot tell them apart. Shorter windows (0.5) would separate the two.
- Forcing each window to start exactly at the previous prediction makes one bad window fatal. Letting windows that contain data correct their own starting point is a likely improvement.
- The gap test used one starting point per spacing, so the line between 1 and 1.5 is approximate.
- T = 10 was not run. With data in every window, the error at window ends stayed flat over five windows, so longer T is expected to behave the same.

16.8 Files

Code: `scripts/time_marching.py`. Example: `python time_marching.py --T 5 --intervals "0.475-0.525, 1.475-1.525" --csv runs.csv`, or from Python, `Lorenz_Time_Marching(T=5, data_intervals=[(0.475, 0.525), ...]).run()`. Results: `results/horizon/` (stage1_full_data.csv, stage1_time_marching.csv, stage2_sampling.csv, and one prediction table per run).